Todo Management API

DSS2 Project Report

RESTful API Backend with FastAPI + PostgreSQL

Technologies: Python / FastAPI / PostgreSQL / Redis / RabbitMQ / Docker
Port: 3087 | API Version: 1.0.0
Academic Year: 2025-2026

1. Project Overview

This project consists of building a RESTful Todo management API as part of the DSS2 course. The API exposes endpoints for user authentication, public read-only access to todos, and full CRUD operations for authenticated users' private todos.

The solution is fully containerized via Docker and complies with the project specifications, including listening on port 3087.

1.1 Technology Stack

Component	Technology	Version
API Framework	FastAPI	0.111.0
ASGI Server	Uvicorn	0.30.1
Database	PostgreSQL	16 (Alpine)
ORM	SQLAlchemy	2.0.30
Authentication	JWT (python-jose)	3.3.0
Password Hashing	passlib + bcrypt	1.7.4 / 4.0.1
Validation	Pydantic	2.7.4
Cache	Redis	7 (Alpine)
Message Broker	RabbitMQ	3 (management)
Containerization	Docker + Compose	Latest

2. Running the Project Locally

2.1 Prerequisites

- Docker Desktop installed and running
- Python 3.12 (optional, for running outside Docker)
- Project files in a folder without special characters (e.g. C:\files2)

2.2 Full Docker Launch

This method starts the API, PostgreSQL database, Redis and RabbitMQ simultaneously.

```
cd C:\files2
docker compose up --build
```

The API is available at: http://localhost:3087

The Swagger documentation is available at: http://localhost:3087/docs

2.3 Local Launch

To run only the database via Docker and the API locally:

```
# Terminal 1 - start the DB
cd C:\files2 && docker compose up db

# Terminal 2 - start the API
pip install -r requirements.txt
python run_local.py
```

2.4 Environment Variables

Variable	Default Value	Description
DATABASE_URL	postgresql://todo_user:todo_password@db:5432/todo_db	PostgreSQL connection URL
SECRET_KEY	change-this-in-production	JWT secret key
REDIS_URL	redis://redis:6379	Redis connection URL
RABBITMQ_URL	amqp://guest:guest@rabbitmq:5672/	RabbitMQ connection URL

3. Database and Schema

3.1 Configuration

The project uses PostgreSQL 16 as the primary database. Tables are created automatically at application startup via SQLAlchemy (Base.metadata.create_all).

3.2 Tables

Table	Description
users	Stores user accounts (unique email, hashed password, displayName)
todo_items	Stores todos with their properties (title, details, priority, due date, status)

3.3 Users Table Schema

Column	Type	Constraints
id	UUID	PRIMARY KEY, default uuid4()
email	VARCHAR(254)	UNIQUE, NOT NULL, INDEX
password_hash	VARCHAR	NOT NULL
display_name	VARCHAR(100)	NULLABLE
created_at	DATETIME	default utcnow()

3.4 Todo Items Table Schema

Column	Type	Constraints
id	UUID	PRIMARY KEY, default uuid4()
user_id	UUID	FOREIGN KEY -> users.id, NOT NULL
title	VARCHAR(100)	NOT NULL
details	VARCHAR(1000)	NULLABLE
priority	ENUM(low/medium/high)	NOT NULL, default medium
due_date	VARCHAR(10)	NULLABLE, format YYYY-MM-DD
is_completed	BOOLEAN	default False, NOT NULL
is_public	BOOLEAN	default False, NOT NULL
created_at	DATETIME	default utcnow()
updated_at	DATETIME	default utcnow(), onupdate

4. API Endpoints

4.1 Authentication

Method	Endpoint	Auth	Description	Codes
POST	/api/auth/register	No	Register a new user	201, 400, 409
POST	/api/auth/login	No	Login and obtain JWT token	200, 400, 401

4.2 Todos

Method	Endpoint	Auth	Description	Codes
GET	/api/todos/public	No	List public todos (paginated)	200
GET	/api/todos	Yes	List authenticated user's todos	200, 401
POST	/api/todos	Yes	Create a new todo	201, 400, 401
GET	/api/todos/{id}	Yes	Get a todo by ID	200, 401, 403, 404
PUT	/api/todos/{id}	Yes	Update a todo (full replacement)	200, 400, 401, 403, 404
PATCH	/api/todos/{id}/completion	Yes	Mark as completed/uncompleted	200, 400, 401, 403, 404
DELETE	/api/todos/{id}	Yes	Delete a todo	204, 401, 403, 404

4.3 Integrations (bonus)

Method	Endpoint	Description
GET	/api/integrations/redis/health	Check Redis connection
GET	/api/integrations/rabbitmq/health	Check RabbitMQ connection

5. Request and Response Examples

5.1 Register — POST /api/auth/register

Request:

```
{ "email": "test@example.com",
"password": "password123",
"displayName": "Test User" }
```

Response 201:

```
{ "id": "c08e7a11-...",
"email": "test@example.com",
"displayName": "Test User" }
```

Response 409 (email already in use):

```
{ "type": "https://httpstatuses.com/409",
"title": "Conflict",
"status": 409,
"errors": { "email": ["Email already in use."] } }
```

5.2 Login — POST /api/auth/login

Request:

```
{ "email": "test@example.com",
"password":
"password123" }
```

Response 200:

```
{ "accessToken": "eyJhbGci...",
"tokenType": "Bearer",
"expiresInSeconds": 3600,
"user": { "id": "...",
"email": "..." } }
```

5.3 Create a Todo — POST /api/todos

Required header: Authorization: Bearer <token>

Request:

```
{ "title": "Buy milk",
"details": "Go to the supermarket",
"priority": "medium",
"dueDate": "2026-04-10",
"isPublic": true }
```

Response 201:

```
{ "id": "1b068f59-...",
"title": "Buy milk",
"priority": "medium",
"dueDate": "2026-04-10",
"isCompleted": false,
"isPublic": true, ... }
```

5.4 List Todos — GET /api/todos

Response 200 (paginated format):

```
{ "items": [{ "id": "...",
"title": "Buy milk", ... }],
"page": 1, "pageSize": 10,
"totalItems": 1, "totalPages": 1 }
```

5.5 Filtering and Sorting Parameters

Parameter	Type	Default	Description
page	int	1	Page number (min 1)
pageSize	int	10	Page size (1-50)
status	string	all	Filter: all | active | completed
priority	string	-	Filter: low | medium | high
dueFrom	date	-	Due date >= dueFrom (YYYY-MM-DD)
dueTo	date	-	Due date <= dueTo (YYYY-MM-DD)
sortBy	string	createdAt	Sort by: createdAt | dueDate | priority | title
sortDir	string	desc	Direction: asc | desc
search	string	-	Keyword search in title and details

6. Bonus Integrations

6.1 Redis

Redis is used to cache public todos. Each request to GET /api/todos/public is cached with a unique key based on all query parameters (page, pageSize, status, priority, etc.).

- Cache TTL: 60 seconds
- Automatic cache invalidation on creation, update or deletion of a public todo
- Health endpoint: GET /api/integrations/redis/health

Redis health response: {"status":"connected","service":"redis"}

6.2 RabbitMQ

RabbitMQ is used to publish domain events. Each CRUD operation publishes a message to the 'todo_events' exchange (fanout type, durable).

Event	Trigger
TodoCreated	Todo creation (POST /api/todos)
TodoUpdated	Todo update (PUT /api/todos/{id})
TodoCompleted	Todo marked as completed (PATCH)
TodoUncompleted	Todo marked as uncompleted (PATCH)
TodoDeleted	Todo deletion (DELETE)

- Exchange: todo_events (fanout, durable)
- Persistent messages (delivery_mode=2)
- Health endpoint: GET /api/integrations/rabbitmq/health

RabbitMQ health response: {"status":"connected","service":"rabbitmq"}

7. Docker Configuration

7.1 Dockerfile

The Dockerfile uses python:3.12-slim as base image, installs dependencies from requirements.txt and exposes port 3087.

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 3087
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "3087"]
```

7.2 docker-compose.yml Services

Service	Image	Port	Role
db	postgres:16-alpine	5432:5432	PostgreSQL database
redis	redis:7-alpine	6379:6379	Redis cache
rabbitmq	rabbitmq:3-management-alpine	5672, 15672	RabbitMQ message broker
api	build: .	3087:3087	FastAPI application

The API depends on all three other services and waits for their 'healthy' state before starting (depends_on with condition: service_healthy).

8. Swagger Screenshots

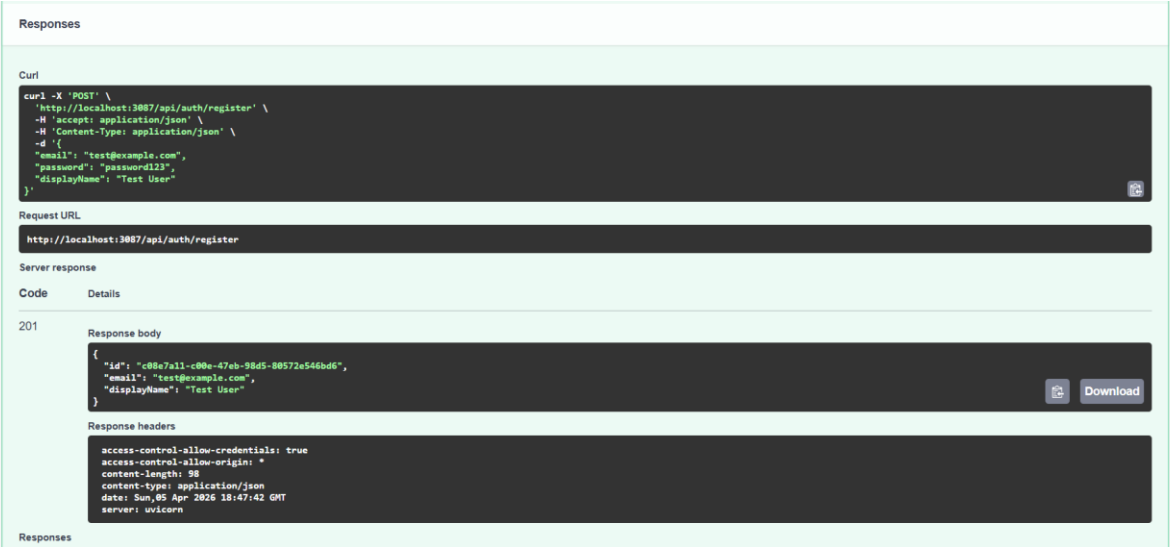
The following screenshots illustrate the different API endpoints tested through the Swagger interface available at http://localhost:3087/docs.

8.1 Swagger Main Page

The Swagger page lists all available endpoints organized in three sections: Authentication, Todos and Integrations.

8.2 Register — 201 Created

Successful user registration returning the created user object (id, email, displayName).



Code	Description	Links
201	Successful Response	No links

Media type
application/json
Controls Accept header.

Example Value | Schema

```
{
  "id": "string",
  "email": "string",
  "displayName": "string"
}
```

422	Validation Error	No links

Media type
application/json

Example Value | Schema

```
{
  "detail": [
    {
      "loc": [
        "string",
        0
      ],
      "msg": "string",
      "type": "string"
    }
  ]
}
```

8.3 Register — 409 Conflict

Attempt to register with an already used email. Returns a 409 error with the message 'Email already in use.' in RFC 7807 format.

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:3087/api/auth/register' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
  "email": "test@example.com",
  "password": "password123",
  "displayName": "Test User"
}'
```

Request URL

```
http://localhost:3087/api/auth/register
```

Server response

Code	Details
409 *Undocumented*	Error: Conflict

Response body

```
{
  "detail": {
    "type": "https://httpstatuses.com/409",
    "title": "Conflict",
    "status": 409,
    "errors": {
      "email": [
        "Email already in use."
      ]
    }
  }
}
```

Download

Response headers

```
access-control-allow-credentials: true
access-control-allow-origin: *
content-length: 127
content-type: application/json
date: Sun,05 Apr 2026 18:59:37 GMT
server: uvicorn
```

Code	Description	Links
201	Successful Response Media type: application/json Controls Accept header. Example Value | Schema	No links

```
{
  "id": "string",
  "email": "string",
  "displayName": "string"
}
```

422	Validation Error Media type: application/json Example Value | Schema	No links

```
{
  "detail": [
    {
      "loc": [
        "string",
        0
      ],
      "msg": "string",
      "type": "string"
    }
  ]
}
```

8.4 Login — 200 OK

Successful login returning the JWT Bearer token, token type, expiration duration (3600 seconds) and user information.

POST /api/auth/login Login

Parameters — Cancel — Reset

No parameters

Request body required — application/json

```
{
  "email": "test@example.com",
  "password": "password123"
}
```

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:3087/api/auth/login' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
  "email": "test@example.com",
  "password": "password123"
}'
```

Request URL

```
http://localhost:3087/api/auth/login
```

Server response

Code	Details
200	Response body

```
{
  "accessToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJjMDh1N2ExMS1jMDB1LTQ3ZWItOThkNS04MDU3MmU1NDZiZDYiLCJleHAiOjE3NzU0MTkzMjB9.FrVT7Tx_CL3AnPi7tNeo9XzVI6ErsEauu49cYeR3djY",
  "tokenType": "Bearer",
  "expiresInSeconds": 3600,
  "user": {
    "id": "c08e7a11-c00e-47eb-98d5-80572e546bd6",
    "email": "test@example.com",
    "displayName": "Test User"
  }
}
```

Download

Response headers

```
access-control-allow-credentials: true
access-control-allow-origin: *
content-length: 334
content-type: application/json
date: Sun,05 Apr 2026 19:01:59 GMT
server: uvicorn
```

Responses

Code	Description	Links
200	Successful Response Media type: application/json Controls Accept header. Example Value | Schema	No links

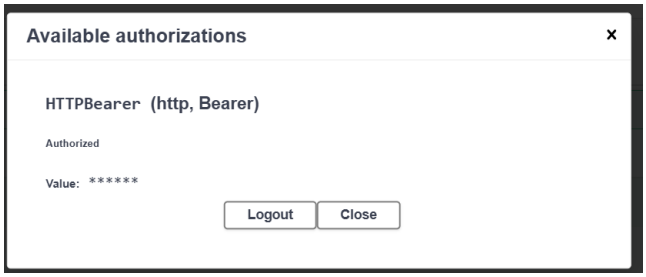
```
{
  "accessToken": "string",
  "tokenType": "Bearer",
  "expiresInSeconds": 3600,
  "user": {
    "id": "string",
    "email": "string",
    "displayName": "string"
  }
}
```

422	Validation Error Media type: application/json Example Value | Schema	No links

```
{
  "detail": [
    {
      "loc": [
        "string",
        0
      ],
      "msg": "string",
      "type": "string"
    }
  ]
}
```

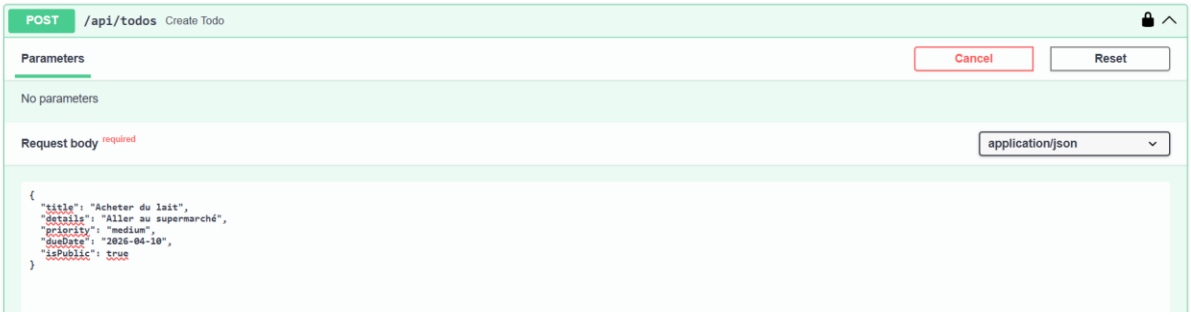
8.5 Authorization in Swagger

The token obtained during login is entered in the Swagger authorization window to access protected endpoints.



8.6 Create Todo — 201 Created

Creation of a todo with title, details, priority, due date and public visibility. The Location header is returned in the response.



Responses

Curl

```
curl -X 'POST' \
  'http://localhost:3087/api/todos' \
  -H 'accept: application/json' \
  -H 'Authorization: Bearer BearereyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJjMDhlN2ExMS1jMDB1LTQ3ZWItOThkNS04MDU3MmU1NDZiZDYiLCJleHAiOjE3NzU0MTkzMj89.FrVT7Tx_CL3AnPi7tNeo9XzVI6ErsEauu49cYeR3djY' \
  -H 'Content-Type: application/json' \
  -d '{
  "title": "Acheter du lait",
  "details": "Aller au supermarché",
  "priority": "medium",
  "dueDate": "2026-04-10",
  "isPublic": true
}'
```

Request URL

```
http://localhost:3087/api/todos
```

Server response

Code	Details
401 *Undocumented*	Error: Unauthorized

Response body

```
{
  "detail": "Invalid token"
}
```

Download

Response headers

```
access-control-allow-credentials: true
access-control-allow-origin: *
content-length: 26
content-type: application/json
date: Sun,05 Apr 2026 19:08:19 GMT
server: uvicorn
```

Responses

Code	Description	Links
201	Successful Response	*No links*

Media type

application/json

Controls `Accept` header.

Example Value | Schema

```
{
  "id": "string",
  "title": "string",
  "details": "string",
  "priority": "string",
  "dueDate": "string",
  "isCompleted": true,
  "isPublic": true,
  "createdAt": "string",
  "updatedAt": "string"
}
```

422	Validation Error	*No links*

Media type

application/json

Example Value | Schema

```
{
  "detail": [
    {
      "loc": [
        "string",
        0
      ],
      "msg": "string",
      "type": "string"
    }
  ]
}
```

8.7 List Todos — 200 OK

Paginated list of the user's todos with pagination metadata (page, pageSize, totalItems, totalPages).

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:3087/api/todos?page=1&pageSize=10&status=all&sortBy=createdAt&sortDir=desc' \
  -H 'accept: application/json' \
  -H 'Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJjMDhlN2ExMS1jMDBlLTQ3ZWItOThkNS04MDU3MmU1NDZiZDYiLCJleHAiOjE3NzU0MTk5NDZ9.Cb-CDM1kLAC9jf8OIirCqfdzE9OV2Rj0hQRbH0x1TIU'
```

Request URL

```
http://localhost:3087/api/todos?page=1&pageSize=10&status=all&sortBy=createdAt&sortDir=desc
```

Server response

Code	Details
200	Response body


```
{
  "items": [
    {
      "id": "1b068f59-6452-48d0-ada3-dba7619afdc6",
      "title": "Acheter du lait",
      "details": "Aller au supermarché",
      "priority": "medium",
      "dueDate": "2026-04-10",
      "isCompleted": false,
      "isPublic": true,
      "createdAt": "2026-04-05T19:12:47.463791Z",
      "updatedAt": "2026-04-05T19:12:47.463795Z"
    }
  ],
  "page": 1,
  "pageSize": 10,
  "totalItems": 1,
  "totalPages": 1
}
```


Download

Response headers

```
content-length: 333
content-type: application/json
date: Sun,05 Apr 2026 19:14:40 GMT
server: uvicorn
```

Responses

Code	Description	Links
200	Successful Response	No links

Media type: application/json

Controls Accept header.

Example Value | Schema


```
{
  "items": [
    {
      "id": "string",
      "title": "string",
      "details": "string",
      "priority": "string",
      "dueDate": "string",
      "isCompleted": true,
      "isPublic": true,
      "createdAt": "string",
      "updatedAt": "string"
    }
  ],
  "page": 0,
  "pageSize": 0,
  "totalItems": 0,
  "totalPages": 0
}
```


422	Validation Error	No links

Media type: application/json

Example Value | Schema


```
{
  "detail": [
    {
      "loc": [
        "string",
        0
      ],
      "msg": "string",
      "type": "string"
    }
  ]
}
```

8.8 Public Todos — 200 OK

List of public todos accessible without authentication. Supports the same filtering and sorting parameters as the private endpoint.

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:3087/api/todos/public?page=1&pageSize=10&status=all&sortBy=createdAt&sortDir=desc' \
  -H 'accept: application/json'
```

Request URL

```
http://localhost:3087/api/todos/public?page=1&pageSize=10&status=all&sortBy=createdAt&sortDir=desc
```

Server response

Code	Details
200	Response body


```
{
  "items": [
    {
      "id": "1b068f59-6452-48d0-ada3-dba7619afdc6",
      "title": "Acheter du lait",
      "details": "Aller au supermarché",
      "priority": "medium",
      "dueDate": "2026-04-10",
      "isCompleted": false,
      "isPublic": true,
      "createdAt": "2026-04-05T19:12:47.463791Z",
      "updatedAt": "2026-04-05T19:12:47.463795Z"
    }
  ],
  "page": 1,
  "pageSize": 10,
  "totalItems": 1,
  "totalPages": 1
}
```


Download

Response headers

```
content-length: 333
content-type: application/json
date: Sun,05 Apr 2026 19:16:07 GMT
server: uvicorn
```

Responses

Code	Description	Links
200	Successful Response	No links

Media type

application/json

Controls Accept header.

Example Value | Schema


```
{
  "items": [
    {
      "id": "string",
      "title": "string",
      "details": "string",
      "priority": "string",
      "dueDate": "string",
      "isCompleted": true,
      "isPublic": true,
      "createdAt": "string",
      "updatedAt": "string"
    }
  ],
  "page": 0,
  "pageSize": 0,
  "totalItems": 0,
  "totalPages": 0
}
```


422	Validation Error	No links

Media type

application/json

Example Value | Schema


```
{
  "detail": [
    {
      "loc": [
        "string",
        0
      ],
      "msg": "string",
      "type": "string"
    }
  ]
}
```

8.9 Update — 200 OK

Full update of a todo (PUT) with modification of the title, details, priority and due date.

PUT /api/todos/{todo_id} Update Todo

Parameters

Cancel Reset

Name	Description
todo_id * required string (path)	1b068f59-6452-48d0-ada3-dba7619afdc6

Request body required

application/json

```
{
  "title": "Acheter du lait et du pain",
  "details": "Supermarché + boulangerie",
  "priority": "high",
  "dueDate": "2026-04-11",
  "isPublic": true,
  "isCompleted": false
}
```

Responses

Curl

```
curl -X 'PUT' \
  'http://localhost:3087/api/todos/1b068f59-6452-48d0-ada3-dba7619afdc6' \
  -H 'accept: application/json' \
  -H 'Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJjMDh1N2ExMS1jMD81LTQ3ZWItOThkNS04MDU3MmU1NDZiZDYiLCJleHAiOjE3NzU0MTk5NDZ9.Cb-CDM1kLAC9jf8OIirCqfdzE9OV2Rj0hQRbH0x1TIU' \
  -H 'Content-Type: application/json' \
  -d '{
  "title": "Acheter du lait et du pain",
  "details": "Supermarché + boulangerie",
  "priority": "high",
  "dueDate": "2026-04-11",
  "isPublic": true,
  "isCompleted": false
}'
```

Request URL

http://localhost:3087/api/todos/1b068f59-6452-48d0-ada3-dba7619afdc6

Server response

Code	Details
200	Response body

```
{
  "id": "1b068f59-6452-48d0-ada3-dba7619afdc6",
  "title": "Acheter du lait et du pain",
  "details": "Supermarché + boulangerie",
  "priority": "high",
  "dueDate": "2026-04-11",
  "isCompleted": false,
  "isPublic": true,
  "createdAt": "2026-04-05T19:12:47.463791Z",
  "updatedAt": "2026-04-05T19:18:34.781855Z"
}
```

Download

Response headers

```
access-control-allow-credentials: true
access-control-allow-origin: *
content-length: 282
content-type: application/json
date: Sun,05 Apr 2026 19:18:33 GMT
server: uvicorn
```

Code	Description	Links
200	Successful Response	No links

Media type

application/json

Controls Accept header.

Example Value | Schema

```
{
  "id": "string",
  "title": "string",
  "details": "string",
  "priority": "string",
  "dueDate": "string",
  "isCompleted": true,
  "isPublic": true,
  "createdAt": "string",
  "updatedAt": "string"
}
```

422	Validation Error	No links

Media type

application/json

Example Value | Schema

```
{
  "detail": [
    {
      "loc": [
        "string",
        0
      ],
      "msg": "string",
      "type": "string"
    }
  ]
}
```

8.10 Completion — 200 OK

Partial update of a todo's completion status (PATCH). Returns the updated todo with isCompleted: true.

PATCH /api/todos/{todo_id}/completion Set Completion

Parameters

Cancel

Name	Description
todo_id * required string (path)	1b068f59-6452-48d0-ada3-dba7619afdc6

Request body required

application/json

```
{
  "isCompleted": true
}
```

Responses

Curl

```
curl -X 'PATCH' \
  'http://localhost:3087/api/todos/1b068f59-6452-48d0-ada3-dba7619afdc6/completion' \
  -H 'accept: application/json' \
  -H 'Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJjMDhlN2ExMS1jMDB1LTQ3ZWItOThkNS04MDU3MmU1NDZiZDYiLCJleHAiOjE3NzU0MTk5NDZ9.Cb-CDM1kLAC9jf8OIirCqfdzE9OV2Rj0hQRbH0x1TIU' \
  -H 'Content-Type: application/json' \
  -d '{
  "isCompleted": true
}'
```

Request URL

```
http://localhost:3087/api/todos/1b068f59-6452-48d0-ada3-dba7619afdc6/completion
```

Server response

Code	Details
200	Response body

```
{
  "id": "1b068f59-6452-48d0-ada3-dba7619afdc6",
  "title": "Acheter du lait et du pain",
  "details": "Supermarché + boulangerie",
  "priority": "high",
  "dueDate": "2026-04-11",
  "isCompleted": true,
  "isPublic": true,
  "createdAt": "2026-04-05T19:12:47.463791Z",
  "updatedAt": "2026-04-05T19:20:45.709697Z"
}
```

Download

Response headers

```
access-control-allow-credentials: true
access-control-allow-origin: *
content-length: 281
content-type: application/json
date: Sun,05 Apr 2026 19:20:45 GMT
server: uvicorn
```

Responses

Code	Description	Links
200	Successful Response	No links

Media type

application/json

Controls Accept header.

Example Value | Schema

```
{
  "id": "string",
  "title": "string",
  "details": "string",
  "priority": "string",
  "dueDate": "string",
  "isCompleted": true,
  "isPublic": true,
  "createdAt": "string",
  "updatedAt": "string"
}
```

422	Validation Error	No links

Media type

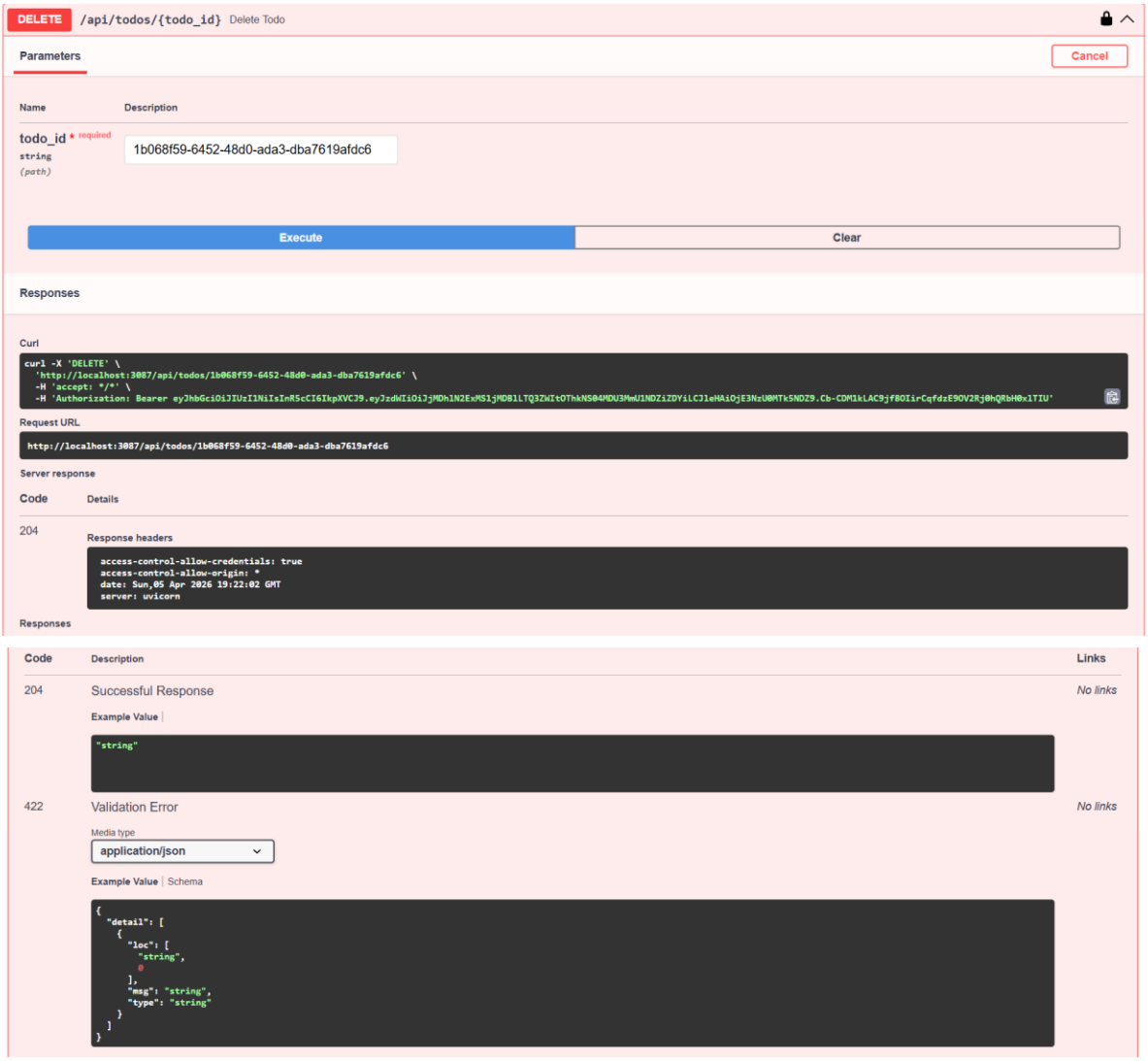
application/json

Example Value | Schema

```
{
  "detail": [
    {
      "loc": [
        "string",
        0
      ],
      "msg": "string",
      "type": "string"
    }
  ]
}
```

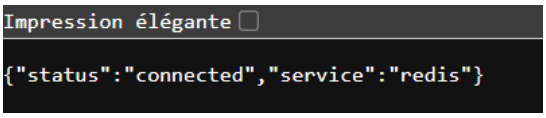
8.11 Delete — 204 No Content

Successful deletion of a todo. Returns a 204 code with no body, in accordance with the specifications.



8.12 Redis Health

The Redis health endpoint confirms the connection: {"status":"connected","service":"redis"}

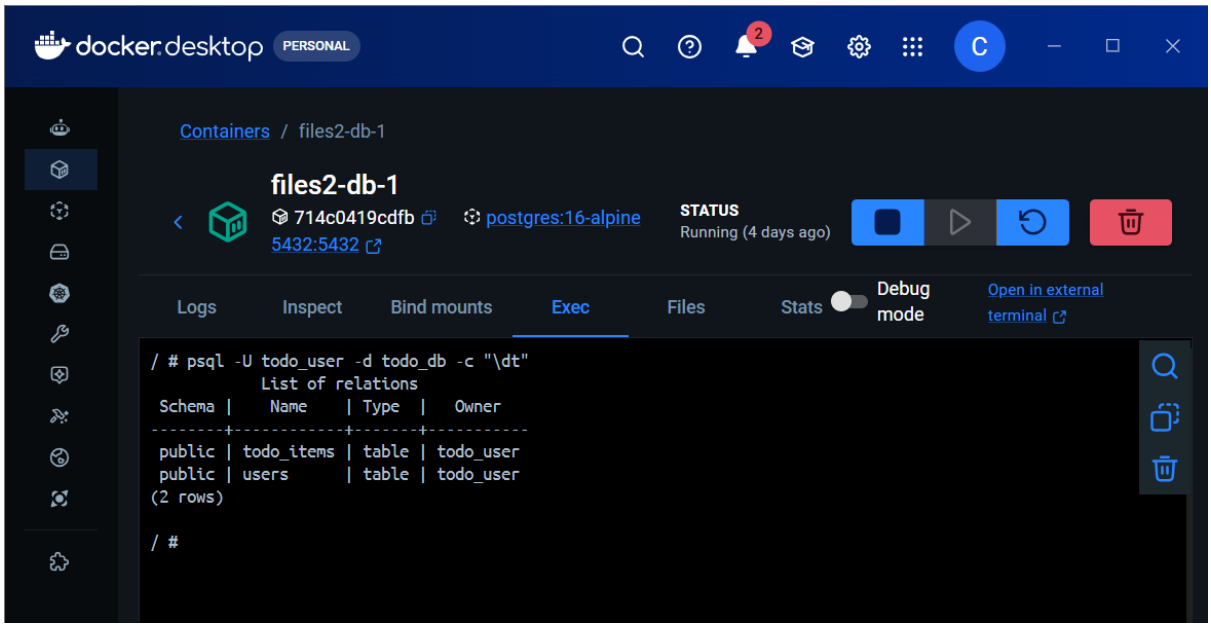


8.13 RabbitMQ Health

The RabbitMQ health endpoint confirms the connection: {"status":"connected","service":"rabbitmq"}

8.14 Database Tables

The two tables automatically created at startup are visible in PostgreSQL: todo_items and users, both owned by the todo_user



9. Problems Encountered and Solutions

During the development and deployment of this project, several technical issues were encountered. Below is a detailed summary of each problem and the solution implemented.

9.1 bcrypt 5.x incompatibility with passlib 1.7.4

Problem: On the first Docker launch, every request to POST /api/auth/register returned a 500 Internal Server Error. The Docker logs showed two distinct errors:

- AttributeError: module 'bcrypt' has no attribute '__about__' — passlib 1.7.4 tries to read the bcrypt version via a '__about__' attribute that no longer exists in bcrypt 5.x
- ValueError: password cannot be longer than 72 bytes — bcrypt 5.x adds strict password length validation that passlib does not handle correctly

Cause: pip automatically installed bcrypt version 5.0.0 (latest available), which is incompatible with passlib 1.7.4, an older unmaintained library.

Solution: Explicitly pin bcrypt to version 4.0.1 in requirements.txt. Version 4.x is the last version compatible with passlib 1.7.4. After adding bcrypt==4.0.1 and rebuilding the Docker image (docker compose up --build), the problem was resolved.

9.2 UnicodeDecodeError when running locally via Spyder

Problem: When attempting to launch the API locally via Spyder (run_local.py), a UnicodeDecodeError: 'utf-8' codec can't decode byte 0xe9 in position 103 appeared consistently when connecting to PostgreSQL via psycopg2.

Cause: This is a known psycopg2 bug on Windows. The library attempts to read a PostgreSQL configuration file (pgpass.conf or pg_service.conf) whose path contains the Windows AppData directory. On this system, the internal Windows system path contained a character encoded in latin-1 (0xe9 = accented e), which psycopg2 tried to decode as UTF-8, causing the error.

Resolution attempts:

- Adding connect_args={'passfile': ''} to create_engine() to ignore the pgpass file — ineffective
- Setting PGPASSFILE, PGSYSCONFDIR, PGSERVICEFILE environment variables pointing to empty files — ineffective
- Setting PYTHONIOENCODING=utf-8 — ineffective

Final solution: Local launch via Spyder was abandoned in favour of a full Docker launch (docker compose up --build), which is not affected by this issue since execution occurs in a Linux container without problematic Windows paths. This approach is also recommended to ensure environment reproducibility.

9.3 Path with special characters in the Windows terminal

Problem: The initial working folder was named 'files (2)' with spaces and parentheses, causing errors in the Windows terminal when running docker compose commands. The terminal did not recognize the path even when quoted in certain contexts.

Solution: The folder was renamed to 'files2' (no spaces or parentheses) and moved to the root of the C: drive (C:\files2) to eliminate any risk related to special paths.

9.4 Docker Desktop not started / Outdated WSL

Problem: On the first launch of Docker Desktop, the application displayed a 'WSL needs updating' error message and the Docker engine failed to start (Engine starting remained stuck).

Solution: WSL was updated using the wsl --update command in the Windows terminal, followed by a WSL restart (wsl --shutdown), then clicking 'Try Again' in Docker Desktop. After these steps, the Docker engine started correctly.

9.5 Installing Python dependencies in the Spyder environment

Problem: Spyder uses its own isolated Python environment (spyder-runtime) separate from the system Python installation. Standard pip and python commands did not work in the Windows terminal because Python was not installed in the system PATH.

Solution: Used the Python embedded in the Spyder environment via its full path: C:\Users\chloe\AppData\Local\spyder-6\envs\spyder-runtime\python.exe -m pip install -r requirements.txt. pip itself had to be installed beforehand via python -m ensurepip.

9.6 Invalid JWT token in Swagger

Problem: During the first Swagger tests, requests to protected endpoints returned a 401 Invalid token error despite entering the token in the Authorize field.

Cause: The token had been entered with the 'Bearer ' prefix, whereas Swagger adds this prefix automatically. Additionally, the token obtained during login had expired between the time of entry and the execution of the request.

Solution: Enter only the raw token (without 'Bearer ') in the Swagger Authorize field, and ensure using a fresh token obtained just before testing.

10. File Structure

File	Role
main.py	FastAPI entry point, CORS middleware, validation error handler
auth.py	Authentication routes (register, login)
auth_utils.py	JWT utilities, password hashing, authentication middleware
todos.py	Todo CRUD routes (list, create, get, update, patch, delete)
models.py	SQLAlchemy models (User, TodoItem, PriorityEnum)
schemas.py	Pydantic schemas for request and response validation
database.py	SQLAlchemy configuration, engine, SessionLocal
integrations.py	Redis helpers (cache) and RabbitMQ (events), health endpoints
Dockerfile	Production Docker image Python 3.12-slim
docker-compose.yml	Orchestration of 4 services (api, db, redis, rabbitmq)
requirements.txt	Pinned Python dependencies
run_local.py	Local launch script (development)

11. Source Code

This section presents the key source files of the project. Each file is accompanied by a description of its role and the main design choices made.

11.1 main.py — Application Entry Point

This file is the FastAPI application entry point. It configures CORS middleware (allowing all origins for Cypress testing), registers the three routers (auth, todos, integrations), and defines a custom validation error handler that formats 400 errors according to the RFC 7807 Problem Details standard expected by the specification.

The lifespan context manager ensures database tables are created automatically on startup via SQLAlchemy's create_all(), removing the need for manual migrations during development.

```python
from fastapi import FastAPI, Request
from fastapi.middleware.cors import CORSMiddleware
from fastapi.exceptions import RequestValidationError
from fastapi.responses import JSONResponse
from contextlib import asynccontextmanager

from database import engine, Base
import auth, todos, integrations


@asynccontextmanager
async def lifespan(app: FastAPI):
    Base.metadata.create_all(bind=engine)
    yield


app = FastAPI(
    title="Todo Management API",
    description="DSS2 Project - RESTful Todo API",
    version="1.0.0",
    lifespan=lifespan,
)

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)


@app.exception_handler(RequestValidationError)
async def validation_exception_handler(request: Request, exc: RequestValidationError):
    errors = {}
    for e in exc.errors():
        # Extract field name from loc tuple
        loc = e.get("loc", [])
        field = str(loc[-1]) if loc else "general"
        errors.setdefault(field, []).append(e["msg"])
    return JSONResponse(
        status_code=400,
        content={
            "type": "https://httpstatuses.com/400",
            "title": "Validation failed",
            "status": 400,
            "errors": errors,
        },
    )
```

11.2 models.py — Database Models

This file defines the two SQLAlchemy ORM models that map directly to the PostgreSQL tables. The User model uses a UUID primary key (auto-generated), stores the email as a unique indexed column, and holds a hashed password — never the plaintext password. The TodoItem model has a foreign key relationship to User with cascade delete, meaning all todos are automatically removed when a user is deleted.

The PriorityEnum is defined as both a Python enum and a PostgreSQL native ENUM type via SAEnum, ensuring database-level validation of the priority field values (low, medium, high).

run_local.py* × auth.py × auth_utils.py × database.py × integrations.py* × main.py × models.py × requirements.txt × schemas.py × todos.py ×

```python
import uuid
from datetime import datetime
from sqlalchemy import Column, String, Boolean, DateTime, ForeignKey, Enum as SAEnum
from sqlalchemy.dialects.postgresql import UUID
from sqlalchemy.orm import relationship
import enum

from database import Base


class PriorityEnum(str, enum.Enum):
    low = "low"
    medium = "medium"
    high = "high"


class User(Base):
    __tablename__ = "users"

    id = Column(UUID(as_uuid=True), primary_key=True, default=uuid.uuid4)
    email = Column(String(254), unique=True, nullable=False, index=True)
    password_hash = Column(String, nullable=False)
    display_name = Column(String(100), nullable=True)
    created_at = Column(DateTime, default=datetime.utcnow)

    todos = relationship("TodoItem", back_populates="user", cascade="all, delete-orphan")


class TodoItem(Base):
    __tablename__ = "todo_items"

    id = Column(UUID(as_uuid=True), primary_key=True, default=uuid.uuid4)
    user_id = Column(UUID(as_uuid=True), ForeignKey("users.id"), nullable=False)
    title = Column(String(100), nullable=False)
    details = Column(String(1000), nullable=True)
    priority = Column(SAEnum(PriorityEnum), nullable=False, default=PriorityEnum.medium)
    due_date = Column(String(10), nullable=True)  # YYYY-MM-DD
    is_completed = Column(Boolean, default=False, nullable=False)
    is_public = Column(Boolean, default=False, nullable=False)
    created_at = Column(DateTime, default=datetime.utcnow)
    updated_at = Column(DateTime, default=datetime.utcnow, onupdate=datetime.utcnow)

    user = relationship("User", back_populates="todos")

```

11.3 auth_utils.py — Authentication Utilities

This file centralizes all authentication logic. It uses passlib with bcrypt (pinned to version 4.0.1 for compatibility) for password hashing and verification. JWT tokens are generated using python-jose with HS256 algorithm and a 60-minute expiration.

The get_current_user dependency function is used by FastAPI's dependency injection system — any route that declares it as a parameter automatically becomes a protected endpoint. If the token is missing, expired or invalid, a 401 Unauthorized is returned before the route handler even executes.

run_local.py* × auth.py × auth_utils.py × database.py × integrations.py* × main.py × models.py × requirements.txt × schemas.py × todos.py ×

```python
import os
from datetime import datetime, timedelta
from passlib.context import CryptContext
from jose import jwt, JWTError
from fastapi import Depends, HTTPException, status
from fastapi.security import HTTPBearer, HTTPAuthorizationCredentials
from sqlalchemy.orm import Session

from database import get_db
import models

SECRET_KEY = os.getenv("SECRET_KEY", "super-secret-key-change-in-production-please")
ALGORITHM = "HS256"
ACCESS_TOKEN_EXPIRE_MINUTES = 60

pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")
bearer_scheme = HTTPBearer()


def hash_password(password: str) -> str:
    return pwd_context.hash(password)


def verify_password(plain: str, hashed: str) -> bool:
    return pwd_context.verify(plain, hashed)


def create_access_token(user_id: str) -> str:
    expire = datetime.utcnow() + timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES)
    payload = {"sub": user_id, "exp": expire}
    return jwt.encode(payload, SECRET_KEY, algorithm=ALGORITHM)


def get_current_user(
    credentials: HTTPAuthorizationCredentials = Depends(bearer_scheme),
    db: Session = Depends(get_db),
) -> models.User:
    token = credentials.credentials
    try:
        payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
        user_id: str = payload.get("sub")
        if user_id is None:
            raise HTTPException(status_code=status.HTTP_401_UNAUTHORIZED, detail="Invalid token")
    except JWTError:
        raise HTTPException(status_code=status.HTTP_401_UNAUTHORIZED, detail="Invalid token")

    user = db.query(models.User).filter(models.User.id == user_id).first()
    if user is None:
        raise HTTPException(status_code=status.HTTP_401_UNAUTHORIZED, detail="User not found")
    return user
```

11.4 todos.py — CRUD Routes

This is the largest and most important file in the project. It implements all seven todo endpoints. Key design decisions include:

- The apply_filters_and_sort() helper centralizes all filtering and sorting logic, avoiding code duplication between the public and private listing endpoints
- The paginate() helper returns the total item count and computes total pages in a single database query
- Cache invalidation (_invalidate_public_cache) uses Redis key pattern deletion to invalidate all cached pages when a public todo changes
- Every mutation (create, update, patch, delete) publishes a domain event to RabbitMQ via publish_event(), but this never blocks the CRUD operation — failures are silently caught
- Ownership checks (str(todo.user_id) != str(current_user.id)) return 403 Forbidden to prevent cross-user access

run_local.py* × | auth.py × | auth_utils.py × | database.py × | integrations.py* × | main.py × | models.py × | requirements.txt × | schemas.py* × | todos.py ×

```python
import math
from datetime import datetime
from typing import Optional
from fastapi import APIRouter, Depends, HTTPException, Query, status, Response
from sqlalchemy.orm import Session
from sqlalchemy import or_, asc, desc

from database import get_db
import models, schemas
from auth_utils import get_current_user
from integrations import cache_get, cache_set, cache_delete_pattern, publish_event

router = APIRouter()

PUBLIC_CACHE_TTL = 60  # seconds


def todo_to_response(todo: models.TodoItem) -> schemas.TodoResponse:
    return schemas.TodoResponse(
        id=str(todo.id),
        title=todo.title,
        details=todo.details,
        priority=todo.priority.value if hasattr(todo.priority, "value") else todo.priority,
        dueDate=todo.due_date,
        isCompleted=todo.is_completed,
        isPublic=todo.is_public,
        createdAt=todo.created_at.isoformat() + "Z" if todo.created_at else None,
        updatedAt=todo.updated_at.isoformat() + "Z" if todo.updated_at else None,
    )


def apply_filters_and_sort(query, model, status_filter, priority, due_from, due_to, sort_by, sort_dir, search):
    if status_filter == "active":
        query = query.filter(model.is_completed == False)
    elif status_filter == "completed":
        query = query.filter(model.is_completed == True)

    if priority:
        query = query.filter(model.priority == priority)

    if due_from:
        query = query.filter(model.due_date >= due_from)
    if due_to:
        query = query.filter(model.due_date <= due_to)

    if search:
        pattern = f"%{search}%"
        query = query.filter(or_(model.title.ilike(pattern), model.details.ilike(pattern)))
```

```python

    sort_col_map = {
        "createdAt": model.created_at,
        "dueDate": model.due_date,
        "priority": model.priority,
        "title": model.title,
    }
    col = sort_col_map.get(sort_by, model.created_at)
    query = query.order_by(asc(col) if sort_dir == "asc" else desc(col))

    return query


def paginate(query, page, page_size):
    total = query.count()
    items = query.offset((page - 1) * page_size).limit(page_size).all()
    total_pages = math.ceil(total / page_size) if total > 0 else 1
    return items, total, total_pages


def _invalidate_public_cache():
    """Delete all cached public todo pages when data changes."""
    cache_delete_pattern("public_todos:*")


# ── Public endpoint ──────────────────────────────────────────────

@router.get("/public", response_model=schemas.PaginatedTodoResponse)
def list_public_todos(
    page: int = Query(1, ge=1),
    pageSize: int = Query(10, ge=1, le=50),
    status: Optional[str] = Query("all"),
    priority: Optional[str] = Query(None),
    dueFrom: Optional[str] = Query(None),
    dueTo: Optional[str] = Query(None),
    sortBy: str = Query("createdAt"),
    sortDir: str = Query("desc"),
    search: Optional[str] = Query(None, max_length=100),
    db: Session = Depends(get_db),
):
    # Build a cache key from all query params
    cache_key = (
        f"public_todos:p{page}:ps{pageSize}:st{status}:pr{priority}"
        f":df{dueFrom}:dt{dueTo}:sb{sortBy}:sd{sortDir}:s{search}"
    )
    cached = cache_get(cache_key)
    if cached:
        return schemas.PaginatedTodoResponse(**cached)

    query = db.query(models.TodoItem).filter(models.TodoItem.is_public == True)
    query = apply_filters_and_sort(query, models.TodoItem, status, priority, dueFrom, dueTo, sortBy, sortDir, search)
    items, total, total_pages = paginate(query, page, pageSize)

    result = schemas.PaginatedTodoResponse(
        items=[todo_to_response(t) for t in items],
        page=page,
        pageSize=pageSize,
        totalItems=total,
        totalPages=total_pages,
    )
    cache_set(cache_key, result.model_dump(), ttl=PUBLIC_CACHE_TTL)
    return result


# ── Authenticated endpoints ──────────────────────────────────────

@router.get("", response_model=schemas.PaginatedTodoResponse)
def list_todos(
    page: int = Query(1, ge=1),
    pageSize: int = Query(10, ge=1, le=50),
    status: Optional[str] = Query("all"),
    priority: Optional[str] = Query(None),
    dueFrom: Optional[str] = Query(None),
    dueTo: Optional[str] = Query(None),
    sortBy: str = Query("createdAt"),
    sortDir: str = Query("desc"),
    search: Optional[str] = Query(None, max_length=100),
    db: Session = Depends(get_db),
    current_user: models.User = Depends(get_current_user),
):
    query = db.query(models.TodoItem).filter(models.TodoItem.user_id == current_user.id)
    query = apply_filters_and_sort(query, models.TodoItem, status, priority, dueFrom, dueTo, sortBy, sortDir, search)
    items, total, total_pages = paginate(query, page, pageSize)
    return schemas.PaginatedTodoResponse(
        items=[todo_to_response(t) for t in items],
        page=page,
        pageSize=pageSize,
        totalItems=total,
        totalPages=total_pages,
    )

```

```python
@router.post("", response_model=schemas.TodoResponse, status_code=201)
def create_todo(
    body: schemas.CreateTodoRequest,
    response: Response,
    db: Session = Depends(get_db),
    current_user: models.User = Depends(get_current_user),
):
    todo = models.TodoItem(
        user_id=current_user.id,
        title=body.title,
        details=body.details,
        priority=body.priority.value,
        due_date=body.dueDate,
        is_public=body.isPublic,
        is_completed=False,
    )
    db.add(todo)
    db.commit()
    db.refresh(todo)
    response.headers["Location"] = f"/api/todos/{todo.id}"

    result = todo_to_response(todo)

    # Publish event + invalidate cache if public
    publish_event("TodoCreated", result.model_dump())
    if todo.is_public:
        _invalidate_public_cache()

    return result


@router.get("/{todo_id}", response_model=schemas.TodoResponse)
def get_todo(
    todo_id: str,
    db: Session = Depends(get_db),
    current_user: models.User = Depends(get_current_user),
):
    todo = db.query(models.TodoItem).filter(models.TodoItem.id == todo_id).first()
    if not todo:
        raise HTTPException(status_code=404, detail="Todo not found")
    if str(todo.user_id) != str(current_user.id):
        raise HTTPException(status_code=403, detail="Forbidden")
    return todo_to_response(todo)

@router.put("/{todo_id}", response_model=schemas.TodoResponse)
def update_todo(
    todo_id: str,
    body: schemas.UpdateTodoRequest,
    db: Session = Depends(get_db),
    current_user: models.User = Depends(get_current_user),
):
    todo = db.query(models.TodoItem).filter(models.TodoItem.id == todo_id).first()
    if not todo:
        raise HTTPException(status_code=404, detail="Todo not found")
    if str(todo.user_id) != str(current_user.id):
        raise HTTPException(status_code=403, detail="Forbidden")

    was_public = todo.is_public
    todo.title = body.title
    todo.details = body.details
    todo.priority = body.priority.value
    todo.due_date = body.dueDate
    todo.is_public = body.isPublic
    todo.is_completed = body.isCompleted
    todo.updated_at = datetime.utcnow()
    db.commit()
    db.refresh(todo)

    result = todo_to_response(todo)
    publish_event("TodoUpdated", result.model_dump())
    if was_public or todo.is_public:
        _invalidate_public_cache()

    return result


@router.patch("/{todo_id}/completion", response_model=schemas.TodoResponse)
def set_completion(
    todo_id: str,
    body: schemas.SetCompletionRequest,
    db: Session = Depends(get_db),
    current_user: models.User = Depends(get_current_user),
):
    todo = db.query(models.TodoItem).filter(models.TodoItem.id == todo_id).first()
    if not todo:
        raise HTTPException(status_code=404, detail="Todo not found")
    if str(todo.user_id) != str(current_user.id):
        raise HTTPException(status_code=403, detail="Forbidden")

    todo.is_completed = body.isCompleted
    todo.updated_at = datetime.utcnow()
    db.commit()
    db.refresh(todo)
```

```python

    result = todo_to_response(todo)
    publish_event("TodoCompleted" if body.isCompleted else "TodoUncompleted", result.model_dump())
    if todo.is_public:
        _invalidate_public_cache()

    return result


@router.delete("/{todo_id}", status_code=204)
def delete_todo(
    todo_id: str,
    db: Session = Depends(get_db),
    current_user: models.User = Depends(get_current_user),
):
    todo = db.query(models.TodoItem).filter(models.TodoItem.id == todo_id).first()
    if not todo:
        raise HTTPException(status_code=404, detail="Todo not found")
    if str(todo.user_id) != str(current_user.id):
        raise HTTPException(status_code=403, detail="Forbidden")

    was_public = todo.is_public
    todo_id_str = str(todo.id)
    db.delete(todo)
    db.commit()

    publish_event("TodoDeleted", {"id": todo_id_str})
    if was_public:
        _invalidate_public_cache()

    return Response(status_code=204)

```

11.5 integrations.py — Redis & RabbitMQ

This file implements the two bonus integrations. Redis helpers (cache_get, cache_set, cache_delete_pattern) use short-lived connections and silently fail on any error, ensuring cache unavailability never breaks CRUD operations. The cache TTL is set to 60 seconds for public todos.

The publish_event() function opens a blocking connection to RabbitMQ, declares the fanout exchange (idempotent — safe to call multiple times), publishes a JSON-encoded message with delivery_mode=2 (persistent), then closes the connection. The 2-second socket timeout prevents long hangs if RabbitMQ is unavailable.

Both health endpoints return 200 with a connected status or 503 with the error detail, allowing the Cypress test suite to verify integration availability.

run_local.py* × auth.py × auth_utils.py × database.py × integrations.py* × main.py × models.py × requirements.txt × schemas.py × todos.py ×

```python
import os
import json
from fastapi import APIRouter
from fastapi.responses import JSONResponse

router = APIRouter()

REDIS_URL = os.getenv("REDIS_URL", "redis://redis:6379")
RABBITMQ_URL = os.getenv("RABBITMQ_URL", "amqp://guest:guest@rabbitmq:5672/")


def get_redis_client():
    import redis
    return redis.from_url(REDIS_URL, socket_connect_timeout=2, decode_responses=True)


def cache_get(key: str):
    """Return parsed JSON value from Redis or None."""
    try:
        r = get_redis_client()
        value = r.get(key)
        if value:
            return json.loads(value)
    except Exception:
        pass
    return None


def cache_set(key: str, value, ttl: int = 60):
    """Store JSON value in Redis with TTL (seconds). Silently fails."""
    try:
        r = get_redis_client()
        r.setex(key, ttl, json.dumps(value))
    except Exception:
        pass


def cache_delete_pattern(pattern: str):
    """Delete all keys matching pattern. Silently fails."""
    try:
        r = get_redis_client()
        keys = r.keys(pattern)
        if keys:
            r.delete(*keys)
    except Exception:
        pass
```

```python
# ── RabbitMQ helpers ──────────────────────────────────────────────────────

def publish_event(event_type: str, payload: dict):

    try:
        import pika
        params = pika.URLParameters(RABBITMQ_URL)
        params.socket_timeout = 2
        connection = pika.BlockingConnection(params)
        channel = connection.channel()
        channel.exchange_declare(exchange="todo_events", exchange_type="fanout", durable=True)
        message = json.dumps({"event": event_type, "data": payload})
        channel.basic_publish(
            exchange="todo_events",
            routing_key="",
            body=message,
            properties=pika.BasicProperties(
                delivery_mode=2,  # persistent
                content_type="application/json",
            ),
        )
        connection.close()
    except Exception:
        pass  # Never block CRUD operations


# ── Health endpoints ──────────────────────────────────────────────────────

@router.get("/redis/health")
def redis_health():
    try:
        r = get_redis_client()
        r.ping()
        return JSONResponse(status_code=200, content={"status": "connected", "service": "redis"})
    except Exception as e:
        return JSONResponse(status_code=503, content={"status": "unavailable", "detail": str(e)})


@router.get("/rabbitmq/health")
def rabbitmq_health():
    try:
        import pika
        params = pika.URLParameters(RABBITMQ_URL)
        params.socket_timeout = 2
        connection = pika.BlockingConnection(params)
        connection.close()
        return JSONResponse(status_code=200, content={"status": "connected", "service": "rabbitmq"})
    except Exception as e:
        return JSONResponse(status_code=503, content={"status": "unavailable", "detail": str(e)})
```

11.6 requirements.txt — Dependencies

All dependencies are pinned to exact versions to ensure reproducible builds. The most notable pinning is bcrypt==4.0.1, which was necessary to resolve an incompatibility with passlib 1.7.4 (see Section 9.1 — Problems Encountered).

run_local.py* × | auth.py × | auth_utils.py × | database.py × | integrations.py* × | main.py × | models.py × | requirements.txt × | schemas.py × | todos.py ×

```requirements.txt
fastapi==0.111.0
uvicorn[standard]==0.30.1
sqlalchemy==2.0.30
psycopg2-binary==2.9.9
passlib[bcrypt]==1.7.4
python-jose[cryptography]==3.3.0
pydantic[email]==2.7.4
redis==5.0.4
pika==1.3.2
alembic==1.13.1
bcrypt==4.0.1
```

11.8 Other codes

run_local.py* × auth.py × auth_utils.py × database.py × integrations.py* × main.py × models.py × requirements.txt × schemas.py × todos.py ×

```python
# -*- coding: utf-8 -*-
"""
Created on Tue Apr  7 09:28:30 2026

@author: chloe
"""
import os

# — Paramètres locaux ————————————————————————————————————————
os.environ.setdefault(
    "DATABASE_URL",
    "postgresql://todo_user:todo_password@localhost:5432/todo_db"
)
os.environ.setdefault("SECRET_KEY", "dev-secret-key-local")

# — Créer les tables si elles n'existent pas encore ——————————————
from database import engine, Base
import models  # noqa: charge les modèles
Base.metadata.create_all(bind=engine)
print("✅ Tables créées / vérifiées.")

# — Démarrer uvicorn ————————————————————————————————————————
import uvicorn
if __name__ == "__main__":
    uvicorn.run("main:app", host="0.0.0.0", port=3087, reload=True)
```

run_local.py* × auth.py × auth_utils.py × database.py × integrations.py* × main.py × models.py × requirements.txt × schemas.py × todos.py ×

```python
from fastapi import APIRouter, Depends, HTTPException, status
from sqlalchemy.orm import Session

from database import get_db
import models, schemas
from auth_utils import hash_password, verify_password, create_access_token, ACCESS_TOKEN_EXPIRE_MINUTES

router = APIRouter()


def user_to_response(user: models.User) -> schemas.AuthUserResponse:
    return schemas.AuthUserResponse(
        id=str(user.id),
        email=user.email,
        displayName=user.display_name,
    )


@router.post("/register", response_model=schemas.AuthUserResponse, status_code=201)
def register(body: schemas.RegisterRequest, db: Session = Depends(get_db)):
    existing = db.query(models.User).filter(models.User.email == body.email).first()
    if existing:
        raise HTTPException(
            status_code=status.HTTP_409_CONFLICT,
            detail={"type": "https://httpstatuses.com/409", "title": "Conflict", "status": 409,
                    "errors": {"email": ["Email already in use."]}},
        )

    user = models.User(
        email=body.email,
        password_hash=hash_password(body.password),
        display_name=body.displayName,
    )
    db.add(user)
    db.commit()
    db.refresh(user)
    return user_to_response(user)


@router.post("/login", response_model=schemas.LoginResponse, status_code=200)
def login(body: schemas.LoginRequest, db: Session = Depends(get_db)):
    user = db.query(models.User).filter(models.User.email == body.email).first()
    if not user or not verify_password(body.password, user.password_hash):
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail={"type": "https://httpstatuses.com/401", "title": "Unauthorized",
                    "status": 401, "errors": {"credentials": ["Invalid email or password."]}},
        )

    token = create_access_token(str(user.id))
    return schemas.LoginResponse(
        accessToken=token,
        tokenType="Bearer",
        expiresInSeconds=ACCESS_TOKEN_EXPIRE_MINUTES * 60,
        user=user_to_response(user),
    )
```

run_local.py* × auth.py × auth_utils.py × database.py × integrations.py* × main.py × models.py × requirements.txt × schemas.py × todos.py ×

```python
import os
from sqlalchemy import create_engine
from sqlalchemy.ext.declarative import declarative_base
from sqlalchemy.orm import sessionmaker

DATABASE_URL = os.getenv(
    "DATABASE_URL",
    "postgresql://todo_user:todo_password@localhost:5432/todo_db"
)

engine = create_engine(DATABASE_URL, connect_args={"passfile": ""})
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
Base = declarative_base()

def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

run_local.py* × auth.py × auth_utils.py × database.py × integrations.py* × main.py × models.py × requirements.txt × schemas.py × todos.py ×

```python
from pydantic import BaseModel, EmailStr, Field, field_validator
from typing import Optional, List
from datetime import datetime
from enum import Enum
import re


class PriorityEnum(str, Enum):
    low = "low"
    medium = "medium"
    high = "high"


# ── Auth ──────────────────────────────────────────────────────────

class RegisterRequest(BaseModel):
    email: EmailStr = Field(..., max_length=254)
    password: str = Field(..., min_length=6, max_length=128)
    displayName: Optional[str] = Field(None, max_length=100)


class LoginRequest(BaseModel):
    email: EmailStr = Field(..., max_length=254)
    password: str = Field(..., min_length=6, max_length=128)


class AuthUserResponse(BaseModel):
    id: str
    email: str
    displayName: Optional[str] = None

    class Config:
        from_attributes = True


class LoginResponse(BaseModel):
    accessToken: str
    tokenType: str = "Bearer"
    expiresInSeconds: int = 3600
    user: AuthUserResponse
```

```python
# ── Todo ──────────────────────────────────────────────────────────

class TodoResponse(BaseModel):
    id: str
    title: str
    details: Optional[str] = None
    priority: str
    dueDate: Optional[str] = None
    isCompleted: bool
    isPublic: bool
    createdAt: str
    updatedAt: str

    class Config:
        from_attributes = True


class CreateTodoRequest(BaseModel):
    title: str = Field(..., min_length=3, max_length=100)
    details: Optional[str] = Field(None, max_length=1000)
    priority: PriorityEnum = PriorityEnum.medium
    dueDate: Optional[str] = None
    isPublic: bool = False

    @field_validator("dueDate")
    @classmethod
    def validate_due_date(cls, v):
        if v is not None:
            if not re.match(r"^\d{4}-\d{2}-\d{2}$", v):
                raise ValueError("dueDate must be in YYYY-MM-DD format")
        return v

class UpdateTodoRequest(BaseModel):
    title: str = Field(..., min_length=3, max_length=100)
    details: Optional[str] = Field(None, max_length=1000)
    priority: PriorityEnum
    dueDate: Optional[str] = None
    isPublic: bool = False
    isCompleted: bool = False

    @field_validator("dueDate")
    @classmethod
    def validate_due_date(cls, v):
        if v is not None:
            if not re.match(r"^\d{4}-\d{2}-\d{2}$", v):
                raise ValueError("dueDate must be in YYYY-MM-DD format")
        return v

class SetCompletionRequest(BaseModel):
    isCompleted: bool
```

```python
# ── Pagination ────────────────────────────────────────────────────

class PaginatedTodoResponse(BaseModel):
    items: List[TodoResponse]
    page: int
    pageSize: int
    totalItems: int
    totalPages: int
```

THE END